

A PROJECT REPORT ON

**Intelligent Information Management for Metadata
Extraction Using Image Recognition Model**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

**Kamlesh Badade
Swaroop Channa
Aditya Deshmukh
Rohil Kadam**

**Exam No: B400050330
Exam No: B400050352
Exam No: B400050371
Exam No: B400050570**



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled
**Intelligent Information Management for Metadata
Extraction Using Image Recognition Model**

Submitted by

Kamlesh Badade
Swaroop Channa
Aditya Deshmukh
Rohil Kadam

Exam No: B400050330
Exam No: B400050352
Exam No: B400050371
Exam No: B400050570

are Bonafide students of this institute and the work has been carried out by them under the supervision of **Dr.R.A.Kulkarni** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. R.A.Kulkarni
Internal Guide
Dept. of Computer Engg.

Dr. G.V.Kale
Head
Dept. of Computer Engg.

Dr. S.T.Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune

Date: 13/05/2025

ACKNOWLEDGEMENT

*It gives us immense pleasure in presenting the project report on **'Intelligent Information Management for Metadata Extraction Using Image Recognition Model.'***

*We would like to take this opportunity to thank **Dr.R.A.Kulkarni** giving me all the help and guidance we needed. We are grateful to her for her kind support. Their valuable suggestions were extremely helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support, suggestions.*

Kamlesh Badade
Swaroop Channa
Aditya Deshmukh
Rohil Kadam
(B.E. Computer Engg.)

ABSTRACT

The "Intelligent Information Management for Metadata Extraction Using Image Recognition Model" project aims to streamline the process of extracting relevant metadata from images, enhancing data organization and retrieval. By leveraging advanced image recognition techniques, the system identifies and categorizes visual content, transforming it into structured metadata that can be easily managed and searched.

In addition to extracting metadata, the system integrates machine learning algorithms to improve accuracy and efficiency in recognizing various image types. This allows for the automatic classification of images based on their content, which is particularly beneficial for industries that rely on vast amounts of visual data, such as e-commerce, healthcare, and digital asset management.

The project not only facilitates better information management but also empowers users with a user-friendly interface to interact with the system. By ensuring seamless integration of image recognition and metadata extraction, the proposed solution significantly. accessibility and utility of visual information, paving the way for more informed decision-making and improved operational workflows.

Contents

1	Introduction	1
1.1	Overview	1
1.2	Objectives.....	2
1.3	Problem Definition.....	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	4
2	Literature Survey	5
2.1	Introduction.....	5
2.2	Literature Review	6
3	Software Requirements Specification	11
3.1	Assumptions and Dependencies.....	11
3.2	Functional Requirements.....	12
3.2.1	System Feature 1: Suicide Risk Detection (Functional Requirement.....	2
3.2.2	System Feature 2: User Interface and Visualization (Functional Requirement)	12
3.3	External Interface Requirements.....	13
3.3.1	User Interfaces.....	13
3.3.2	Hardware Interfaces.....	13
3.3.3	Software Interfaces	12
3.3.4	Communication Interfaces.....	12
3.4	Nonfunctional Requirements	13
3.4.1	Performance Requirements	13
3.4.2	Safety Requirements	13
3.4.3	Security Requirements	13
3.4.4	Software Quality Attributes	14
3.5	System Requirements.....	14
3.5.1	Database Requirements.....	14
3.5.2	Software Requirements (Platform Choice).....	14
3.5.3	Hardware Requirements	15

3.6	Analysis Models: SDLC Model to be applied.....	15
4	System Design	16
4.1	System Architecture.....	16
4.2	Mathematical Model.....	17
4.3	Data Flow Diagrams.....	18
4.4	Entity Relationship Diagram.....	19
4.5	UML Diagram.....	20
4.5.1	Class Diagram.....	20
4.5.2	Use Case Diagram.....	21
4.6	Activity Diagram.....	22
4.7	Sequence Diagram.....	23
5	Project Plan	24
5.1	Project Estimate.....	24
5.1.1	Reconciled Estimates.....	24
5.1.2	Project Resources.....	25
5.2	Risk Management.....	27
5.2.1	Risk Identification.....	27
5.2.2	Risk Analysis.....	27
5.2.3	Overview of Risk Mitigation, Monitoring, Management.....	28
5.3	Project Schedule.....	29
5.3.1	Project Task Set.....	29
5.3.2	Timeline Chart.....	30
5.4	Team Organization.....	30
5.4.1	Team structure.....	30
6	Project Implementation	32
6.1	Overview of Project Modules.....	32
6.2	Tools and Technologies Used.....	34
6.3	Algorithm Details.....	34
7	Software Testing	37
7.1	Types of Testing.....	37
7.2	Test cases & Test Results.....	40
7.2.1	Individual Stock Prediction (VMART).....	40
7.2.2	Nifty Index Prediction.....	42
8	Results	43
8.1	Screen Shots.....	44
9	Conclusions	47
9.1	Conclusions.....	47
9.2	Future Work.....	47
9.3	Applications.....	49

10 Appendix	51
10.1 Appendix A.....	51
10.2 Appendix B.....	54
10.3 Appendix C.....	55
11 References	56

List of Figures

1.1 NLP Working.....1

4.1 System Architecture 16

4.2 Mathematical Model 17

4.3 Data Flow Diagrams 18

4.4 Entity Relationship Diagram..... 19

4.5 Class Diagram..... 20

4.6 Use Case Diagram 21

4.7 Activity Diagram..... 22

4.8 Sequence Diagram..... 23

5.3 Timeline Chart23

6.3 Algorithm Flow Diagram..... 23

10.1 Plagiarism Report..... 55

Chapter 1

Introduction

1.1 Overview

The project titled "Intelligent Information Management for Metadata Extraction Using Image Recognition Model" is aimed at automating the extraction and classification of metadata from book cover images. Traditional methods of cataloging books involve significant manual effort in identifying and recording details such as the title, author, and publisher. This process becomes especially time-consuming and error-prone when dealing with large-scale collections, such as those in libraries, bookstores, or digital archives.. To address these challenges, the proposed system leverages Optical Character Recognition (OCR) and Natural Language Processing (NLP) techniques to extract and organize metadata efficiently. OCR is used to identify and digitize the text present on book covers, while NLP models, including transformer-based architectures like BERT or GPT, are utilized to classify and structure the extracted text into meaningful categories. This intelligent information management system not only reduces manual effort but also improves metadata consistency, accuracy, and accessibility. The project has applications in libraries, academic institutions, publishing houses, and digital archives and offers a scalable solution for organizing large volumes of book-related data. Additionally, it lays the foundation for future enhancements like multilingual support, database integration, and advanced metadata validation.

1.2 Objectives

- i. The primary objectives of this project are:
- ii. Predict the stock price values for the upcoming trading week using a deep learning model built in Python.
- iii. Incorporate custom-engineered features into the model to improve flexibility and performance.
- iv. Forecast the next-day price value of key market indices—Nifty 50, Bank Nifty, Fin Nifty, and Midcap Nifty.
- v. Integrate technical indicators and sentiment analysis to enhance prediction accuracy.
- vi. Support more informed and data-driven investment decisions through reliable forecasting.

1.3 Problem Definition

In today's fast-paced world, emotional well-being often goes unnoticed, leading to increased stress and mental health issues. Detecting human emotions using facial expressions can significantly enhance communication, therapy, and human-computer interaction. However, achieving real-time and accurate emotion recognition from facial cues remains a technical challenge. This project aims to develop a system that accurately identifies and classifies human emotions using image-based facial analysis with machine learning techniques.

1.4 Project Scope & Limitations

- i. The primary goal of this project is to develop an intelligent, image-based system capable of extracting and organizing metadata from book cover images. The system aims to assist libraries, publishers, and digital content platforms by automating the cataloging process through a combination of OCR (Optical Character Recognition) and NLP (Natural Language Processing)..
- . Key components within the project scope include:
 - 1.A web-based user interface for uploading book cover images.
 - 2.Integration of OCR tools (e.g., Tesseract) to extract raw text from the image.
 - 3.Display of structured metadata in formats like JSON or a searchable database.
 - 4.Support for user review and manual correction of extracted data.

1.5 Methodologies of Problem solving

1. Problem Definition and Understanding

- Goal: Clearly define the problem, understand the system requirements , user expectations, and technical constraints.
- Example: “Automatically extract structured metadata such as title, author, and publisher from diverse book cover images using OCR and NLP classification models.”

2. Data Collection and Preprocessing

- Goal: :Gather a diverse set of book cover images and prepare them for OCR and NLP processing.
- Steps: Collect book cover images from open datasets or scrape them from online sources.
- Example: Use LSTM for sequence modeling, possibly extend to Seq2Seq for multi-day forecasts.

Information Management for Metadata Extraction

3. Training, Validation, and Evaluation

Goal:: Select or build a model that can accurately classify and organize the extracted text into metadata fields.

- Steps: Evaluate different NLP models like BERT, RoBERTa, or a custom Transformer.

4. Iteration and Optimization

- Goal Refine the system based on performance results, edge cases, and user fee.
 - Steps:
 - Example: If the model struggles with decorative fonts, use data augmentation or train the OCR on similar styled images; retrain the NLP model with domain-specific corrections.
-

Chapter 2

Literature Survey

2.1 Introduction

There are Three major approaches to text summarization: OCR-Based Extraction, LLM-Based Extraction and Hybrid Approaches. OCR-Based Extraction: OCR is a crucial step in extracting metadata from book covers. Once text is extracted using OCR, it undergoes text preprocessing to clean, normalize, and tokenize the data. This prepares the text for further analysis. Examples of Extractive Summarization: Keyword-Based Extraction: Use keyword extraction techniques to identify important words in the extracted text. o Identify layout structure: Use computer vision techniques to determine the layout of the book cover (e.g., header, footer, body). LLM-Based Extraction: LLMs can be used to extract metadata from book covers. First, fine-tune a pre-trained LLM on book cover data. Then, use the finetuned model to process extracted text and identify metadata elements like title, author, and publisher. This approach leverages the power of LLMs for accurate metadata extraction. Examples of LLM-Based Extraction: BERT: A popular pre-trained LLM that has achieved state-of-the-art results on various NLP tasks, including text classification and named entity recognition. GPT: Another powerful LLM that can be used for text generation and understanding. Hybrid Approaches: OCR extracts text from images, while LLMs process and classify the extracted text. This can be done in separate steps or with end-to-end models that perform both tasks simultaneously.

2.2 Literature Review

2.2 BOOK IMAGE DATA EXTRACTION CHALLENGES Book image data extraction presents several challenges due to the variability of book covers and the limitations of existing techniques: **Image Quality:** Low-resolution, blurry, or noisy images can hinder the accuracy of OCR and other image processing techniques. **Layout Complexity:** Book covers can have diverse layouts, with varying positions and arrangements of title, author, and other metadata elements. **Language Variations:** Book covers may contain text in multiple languages, requiring multilingual OCR and NLP capabilities. **Font Variations:** A wide range of fonts, sizes, and styles can make text extraction difficult. **Noise and Artifacts:** Images may contain noise, artifacts, or other visual disturbances that can interfere with accurate metadata extraction. **Incomplete or Missing Information:** Some book covers may not contain all the desired metadata elements, or information may be partially obscured or missing.

2.3 EXISTING SYSTEMS AND MODELS

Several studies and tools have been developed in the area book image data extraction.

These include: **BookNet Toolkit:** This is an open-source toolkit that provides tools for book cover image classification and metadata extraction. It includes pre-trained models, datasets, and other resources for researchers and developers.

Amazon Textract: This is a service from Amazon Web Services (AWS) that uses machine learning to extract text, tables, and forms from documents, including images. It can be used for book cover image data extraction.

Google Cloud Vision API: This is a similar service from Google Cloud Platform that can be used for image analysis and text extraction.

Chapter 3

Software Requirements Specification

3.1 Assumptions and Dependencies

1. Assumptions

- **Clean Historical Data Available**
It is assumed that the uploaded book cover images are of adequate resolution and clarity to allow accurate OCR processing without requiring extensive manual correction.
- **English Language Metadata**
For The system is initially trained to recognize and classify metadata in English. It is assumed that most book covers submitted will follow this language constraint.
- **Standard Metadata Presence**
It It is assumed that common metadata fields—such as title, author, and publisher—are available and visually present on the book cover.

2. Dependencies

- Libraries and Frameworks

The system depends on external Python libraries such as:

- Natural Language Processing: NLTK, spaCy, TextBlob
- Deep Learning: TensorFlow/Keras or PyTorch
- Preprocessing: Pandas, NumPy
- Visualization: Matplotlib, Seaborn
- Transformers: HuggingFace Transformers (e.g., BERT)
- Frontend-Backend Integration The UI is dependent on seamless communication between the frontend (e.g., React) and the Python
- CSV Data Files The application relies on local or server-side CSV files for loading OHLC and PCA data for both training and inference.

3. Frontend-Backend Integration

The UI is dependent on seamless communication between the frontend (e.g., React) and the Python backend (Flask).

- Dataset Format

The application relies on local or server-side CSV files for loading OHLC and PCA data for both training and inference.

- Hardware/Runtime Environment

The system depends on sufficient computational resources (CPU/GPU and RAM) to train or run prediction models smoothly without lag.

3.2 Functional Requirements

The following are the functional requirements for the proposed system:

3.2.1 System Feature 1: Functional Requirement

- **Description:** The system allows users to upload a book cover image and automatically extracts key metadata (e.g., title, author, publisher) using a combination of OCR and NLP models.
- **Functionalities:**
 - Users can upload a book cover image via a web-based UI.
 - The system uses Tesseract OCR to extract raw text from the uploaded image.
 - The extracted text is then passed through an NLP model (e.g., BERT) to classify metadata into predefined categories. Optionally store flagged text for later analysis.
- **Input:** Book cover image file (JPG, PNG).
- **Output:** Structured metadata fields (e.g., title, author, publisher) in a userfriendly and downloadable format.

3.2.2 System Feature 2: User Interface and Visualization (Functional Requirement)

- **Description:** The system provides a clean and user-friendly interface that allows users to upload book cover images, view extracted metadata and download the results in a structured format.
 - **Functionalities:**
 - Simple UI with an upload button to select book cover images from the local device.
 - Real-time display of extracted metadata fields such as title, author, and publisher.
 - Option to download extracted metadata in JSON or CSV format
- Responsive design for compatibility across desktop and mobile devices.

3.3 External Interface Requirements

The following are the external interface requirements for the proposed system:

3.3.1 User Interfaces

- Simple, accessible Chrome extension or browser-based UI.
- Users will be able to:
 - Upload functionality for scanned book covers or pages (image/format).
 - Automatic display of extracted metadata such as title, author,
 - Editable metadata fields allowing users to confirm or correct extracted information.
 - Search and filter tools to browse and locate cataloged books using metadata attributes.
 - Dashboard for administrators to monitor processing logs, track OCR success rates, and manage model outputs..

3.3.2 Hardware Interfaces

- No specialized hardware is required.
- The system can run on:
 - Standard PCs or laptops with internet access.
 - Server environments for hosted versions (GPU for faster training).
- Recommended minimum:
 - 4 GB RAM, Dual-core CPU, 5 GB storage.

3.3.3 Software Interfaces

- **Operating System:** Compatible with Windows, Linux, or macOS.
- **Backend Environment:** Node.js with Express.js (for server-side logic), Python for OCR and NLP processing.
- **Libraries :** Tesseract OCR, Hugging Face Transformers (e.g., BERT), Pandas, NumPy, Express.js
- **Frontend Stack:** : React.js, HTML/CSS/JavaScript.
- **Browser Support:** Chrome, Firefox

3.3.4 Communication Interfaces

- **Local Deployment**
 - HTTP-based interaction between frontend and backend.
 - Localhost or server IP
- **Data Input/Output**
 - Book cover images uploaded via browser interface.
 - Extracted metadata displayed in-browser, downloadable in JSON format

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The system shall extract metadata from standard book cover images within 5–10 seconds under normal usage.
- The user interface shall respond to image uploads and edits within 1 second for optimal user experience.
- The backend shall efficiently process concurrent requests from multiple users without noticeable performance degradation.

3.4.2 Safety Requirements

- The system does not interact with physical components but ensures safe handling of user-uploaded images and extracted metadata..
- System should prevent data loss and gracefully handle errors.

3.4.3 Security Requirements

- The system shall validate all user inputs, including uploaded images, to prevent injection or malformed data attacks.
- Image uploads must be restricted to valid image formats (e.g., JPG, PNG) and processed in isolated environments to avoid execution vulnerabilities.
- For hosted deployments:
 - HTTPS shall be enforced to secure data transmission.
 - Basic authentication and role-based access control can be implemented if user accounts are supported.
 - No sensitive user data is collected or stored by default.

3.4.4 Software Quality Attributes

Quality Attributes

- **Usability:** Intuitive Chrome extension and visual feedback.
- **Maintainability:** Modular code (model, UI, server separated).
- **Portability:** Cross-platform compatibility.
- **Reliability:** Robust performance with accurate classification.
- **Scalability:** Robust performance with accurate classification.

3.5 System Requirements

3.5.1 Database Requirements

The system does not rely on a traditional database like MySQL or MongoDB. Instead, it uses structured CSV/JSON files to manage historical data. Optionally, a lightweight database like SQLite or Firebase can be introduced for logging prediction history, managing user preferences, or future scalability.

3.5.2 Software Requirements (Platform Choice)

The backend of the system is developed in Python 3.10 or higher. Libraries The backend is implemented using Node.js with Express.js, while OCR and NLP functionalities are handled using Python libraries.

The frontend is built using React.js with standard HTML/CSS and JavaScript. Development is typically performed in environments like VS Code or JupyterLab. For deployment, platforms such as AWS, Google Cloud, or local servers can be used.

3.5.3 Hardware Requirements

The minimum hardware required for running the application includes a dual-core CPU, 4 GB of RAM, and at least 5 GB of free disk space. However, for smoother execution and training of models, it is recommended to use a quad-core processor, 8 GB or more of RAM, and an SSD with at least 10 GB of free space. While a GPU is not mandatory, having an NVIDIA GPU significantly speeds up model training, especially for deep learning tasks.

3.6 Analysis Models: SDLC Model to be applied

The project follows the **Iterative Incremental Model** of the Software Development Life Cycle (SDLC).

Justification:

1. **Iterative Development:** Components like OCR, NLP classification, and the UI 2. are developed in cycles, allowing incremental feature enhancements.
2. **User Feedback:** : Early versions of the system are tested to collect feedback from potential users, enabling continuous refinement.
3. **Flexibility:** New modules such as multilingual support or database integration can be added without redesigning the entire syst.
4. **Collaboration:** Parallel development of the backend (OCR/NLP) and frontend (UI) allows efficient team collaboration and faster delivery.

Chapter 4

System Design

4.1 System Architecture

Figure 4.1 illustrates the system architecture used for prediction in this project.

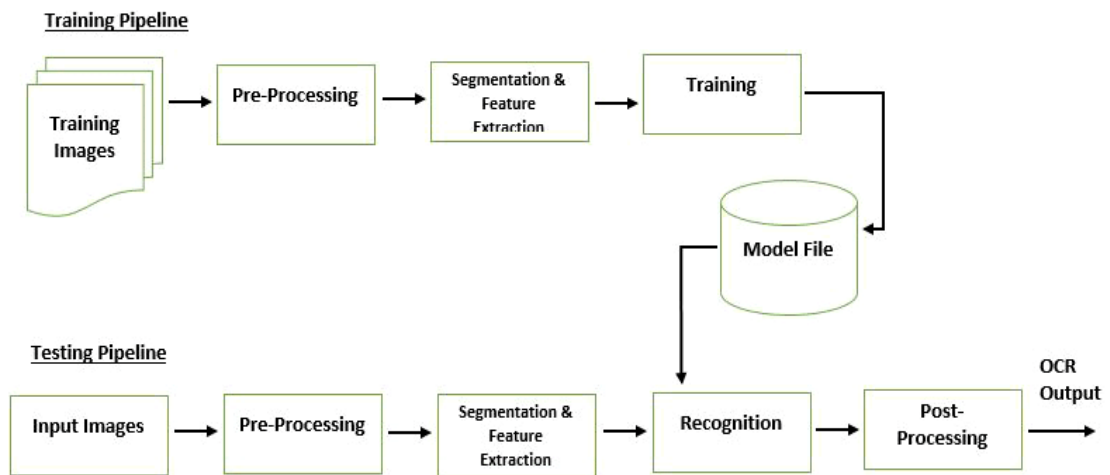


Figure 4.1: System Architecture

4.2 Mathematical Model

Figure 2: Transformers (BERT) model used for metadata.

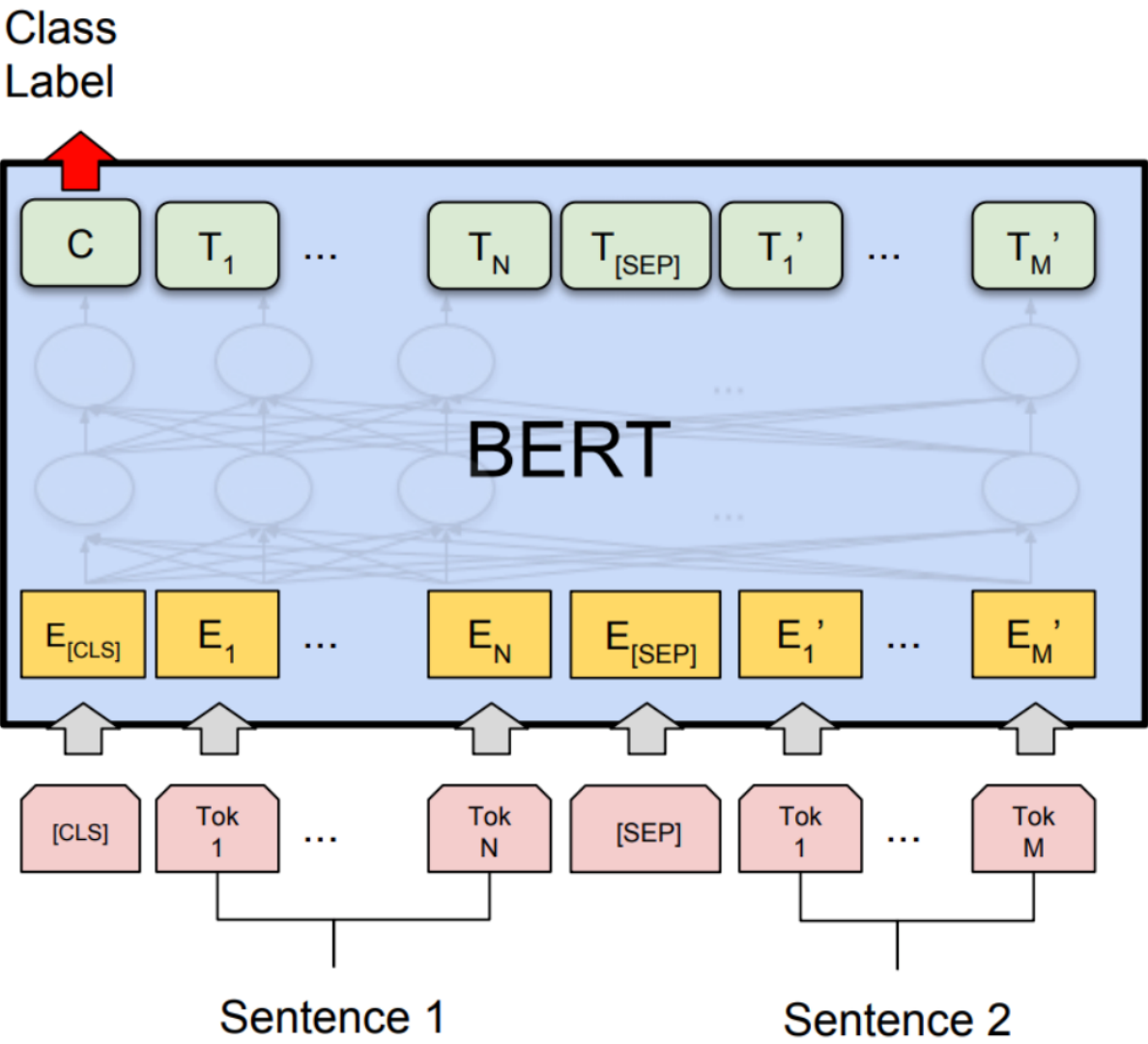


Figure 4.2: DistilBERT Architecture

4.3 Data Flow Diagrams

Figure 4.3 description of each major software function is presented, along with data flow (structured analysis) or class hierarchy (Analysis Class diagram with class description for object-oriented system).

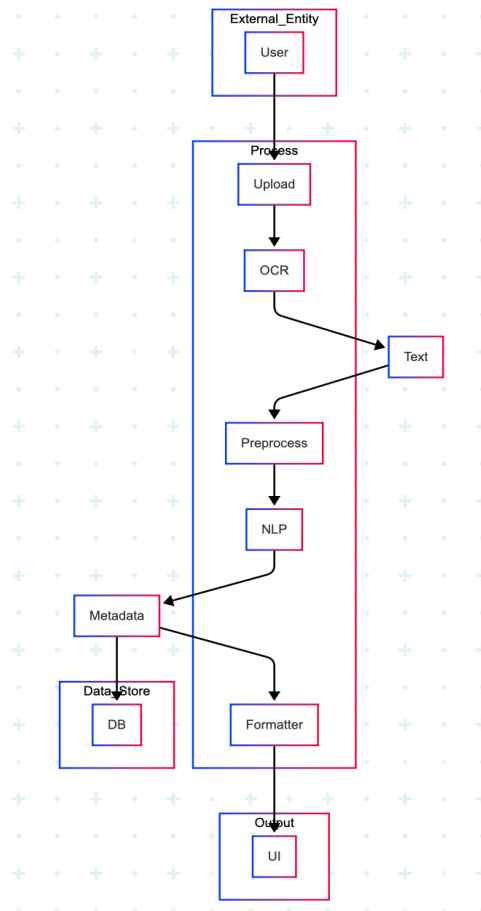


Figure 4.3: Data Flow Diagram

4.4 Entity Relationship Diagram

Figure 4.4 illustrates the Entity Relationship Diagram (ERD) for the proposed system. It outlines the main entities involved and their relationships.

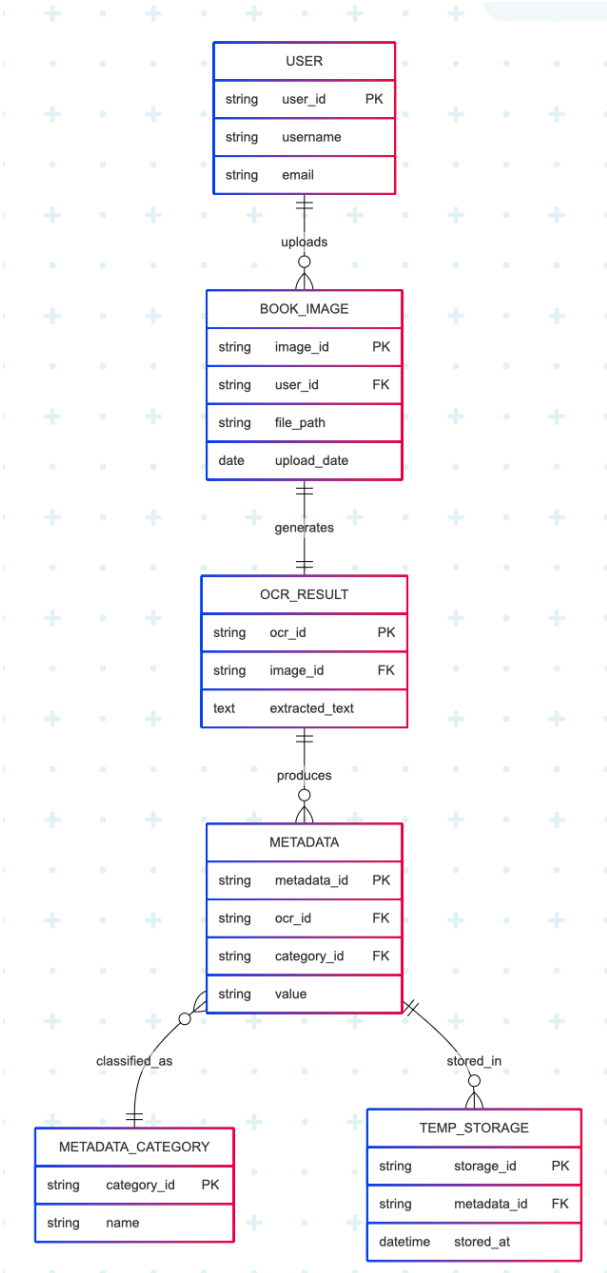


Figure 4.4: Entity Relationship Diagram

4.5 UML Diagram

4.5.1 Class Diagram

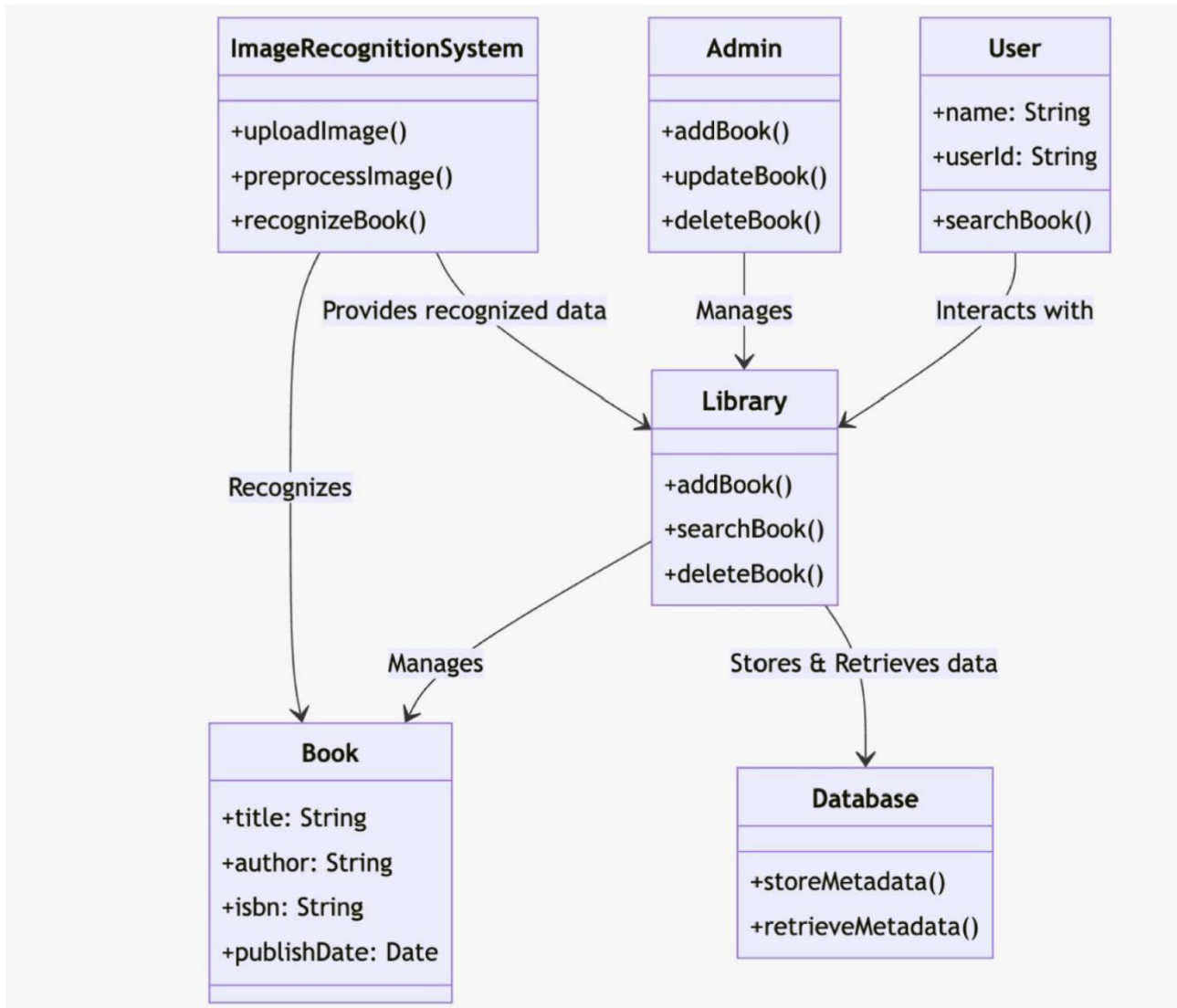


Figure 4.5: Class Diagram

4.5.2 Use Case Diagram

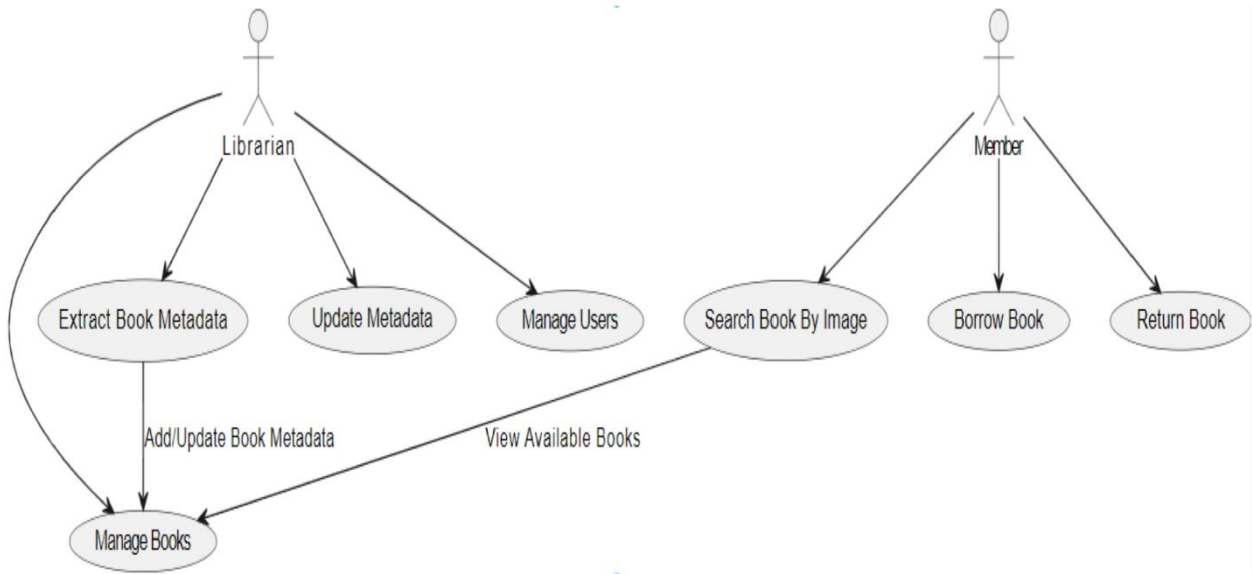


Figure 4.6: Use Case Diagram

4.6 Activity Diagram

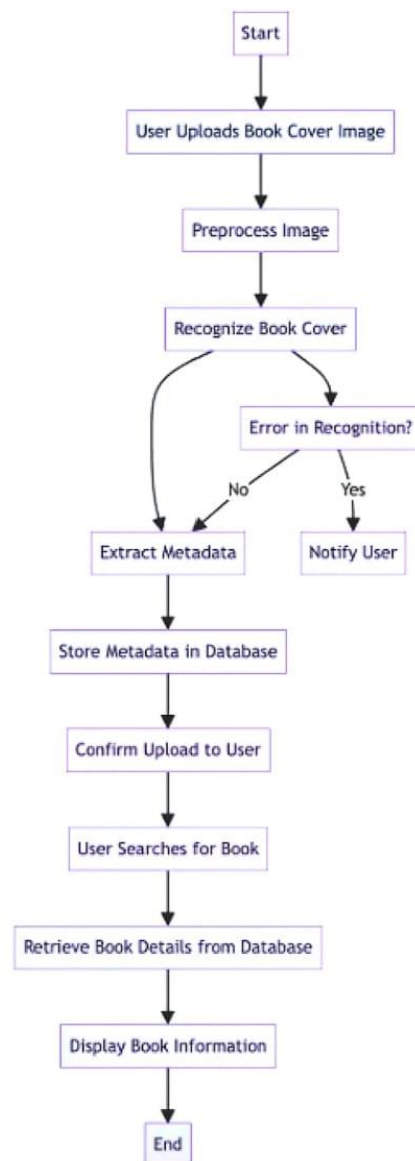


Figure 4.7: Activity Diagram

4.7 Sequence Diagram

Figure 4.8 represents the Sequence Diagram of the proposed system. This diagram models the sequential interaction between different system components over time.

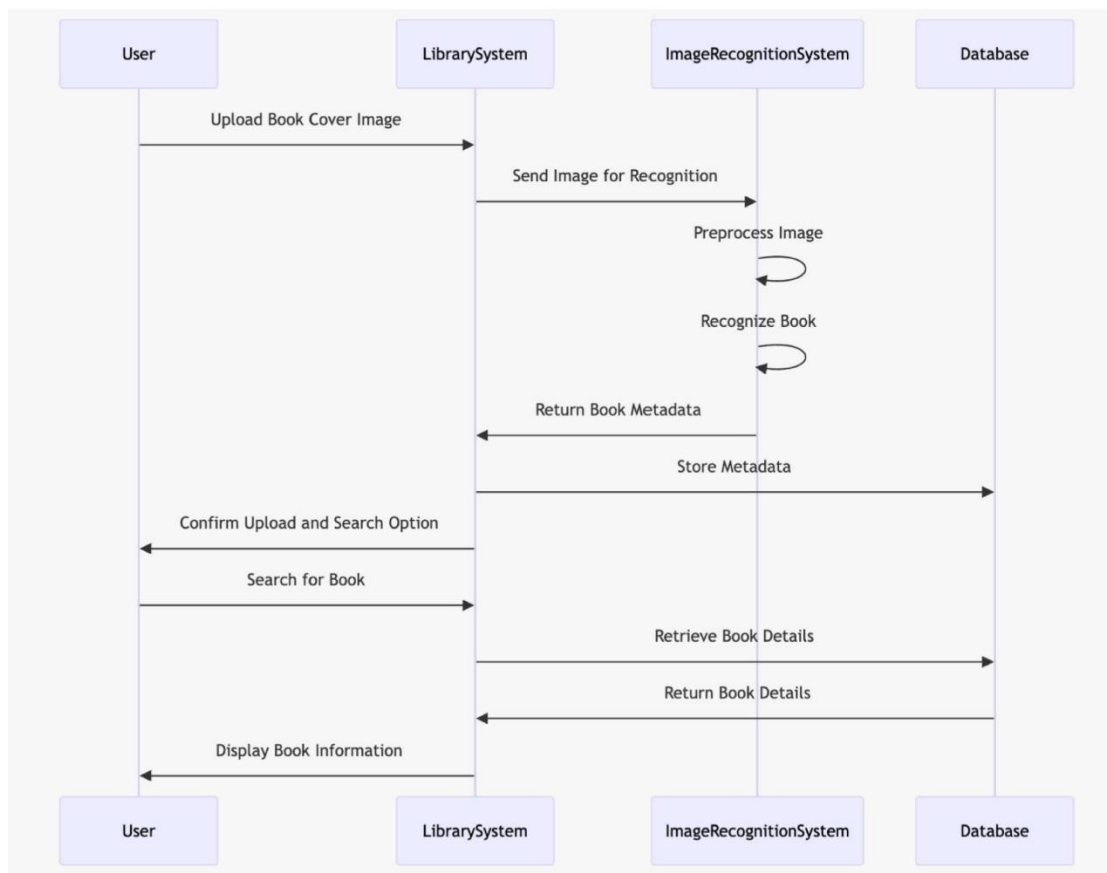


Figure 4.8: Sequence Diagram

Chapter 5

Project Plan

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Project Planning and Preparation (2 weeks):**
Initial planning, team formation, identification of system goals, and selection of tools for image-based metadata extraction using OCR and NLP models.
- **Studying Scope and Feasibility (1 week):**
Evaluating the feasibility of automating metadata extraction from book covers using computer vision and NLP. Assessing dataset availability, OCR limitations, and language constraints.
- **Data Acquisition and Preprocessing (1 month):**
Collecting book cover images and corresponding metadata. Performing image quality checks, resizing, and augmentations. Preprocessing OCR output through text cleaning, normalization, and tokenization.
- **Model Development and Training (2.5 months):**
 - **OCR Integration and Testing (2 weeks):** Integrating Tesseract OCR engine and evaluating output quality across various image types and font styles.
 - **NLP Model Setup and Fine-Tuning (6 weeks):** Implementing and fine-tuning a BERT-based model for metadata classification.

- **Model Evaluation and Optimization (1.5 months):**
Evaluating the classification accuracy using precision, recall, F1 score. Iteratively optimizing preprocessing, OCR parameters.
- **Software Integration and Chrome Extension Development (1 month):**
 - **Frontend Development (2 weeks):**
Building a Chrome extension or simple web interface for real-time text input and result visualization.
 - **OCR and NLP Pipeline Integration (2 weeks):**
Integrating the OCR (Tesseract) and BERT-based classification pipeline into the backend server using Python and Node.js.
- **Testing and Quality Assurance (2 weeks):**
Conducting unit testing, integration testing, and performance testing of all components. Validating OCR accuracy, classification consistency, and end-to-end workflow reliability.
- **Documentation and Presentation (2 weeks):**
Preparing comprehensive technical documentation, user manuals, diagrams (DFD, ERD, UML)
- **Final Review (1 week):**
Performing final checks, gathering feedback from guide/peers

5.1.2 Project Resources

- **Software Development** Designed, developed, and implemented the core backend modules required for the project, including image upload handling, OCR integration, text preprocessing, metadata classification, and JSON output generation. Python was used for model implementation, while Node.js and Express.js were employed for API development. Key libraries included Pandas for data handling, NumPy for array operations, and Tesseract for OCR
- **Natural Language Processing Model:** Built and fine-tuned a BERT-based transformer model for named entity recognition (NER) to identify metadata elements such as book title, author, and publisher from OCR-extracted text. Optimization strategies included model fine-tuning, text normalization, tokenization, and attention-based classification. Pretrained models from Hugging Face were adapted for this task.

- **Data Sources and APIs:** Collected and curated a dataset of book cover images and their associated metadata. Additional open-source datasets were explored for multilingual or genre-diverse samples. Tools such as Label Studio were used for annotation.
- **User Interface and Backend Development:** Developed a responsive and accessible web interface using React.js, HTML5, and CSS3, allowing users to upload book images, preview extracted metadata, and make manual edits. Backend services (OCR, NLP, and database access) were managed using Node.js and Express, with MongoDB serving as a temporary storage layer for metadata and image references.
- **Collaboration and Development Tools:** Utilized GitHub for version control and collaborative development. The team maintained a well-organized repository with regular commits, clearly defined branches, and tracked issues to ensure transparency and coordination.

5.2 Risk Management

5.2.1 Risk Identification

During the risk identification process for our project, we recognized several potential risks that could affect the success and efficiency of the metadata extraction system:

- **Technical Risks:**
 - Inaccuracies in OCR output due to poor image quality or complex book cover designs were addressed by enhancing preprocessing steps and tuning Tesseract parameters.
 - NLP classification errors from misinterpreted or noisy text were reduced by fine-tuning the BERT model and performing robust text normalization.
 - Integration issues between OCR, preprocessing, and NLP modules were minimized through modular design, unit testing, and API based separation of concerns.
 - Performance bottlenecks during simultaneous uploads or model inference were mitigated by optimizing backend routes and using asynchronous processing.

- **Ethical and Privacy Risks:**

- **Misclassification Consequences:** False positives or negatives could lead to incorrect alerts. Risk was addressed with high precision thresholds and confidence-based alert triggering.

- **Operational Risks:**

- **Scope Creep:** Potential delays from miscommunication or scope changes were controlled via weekly progress reviews and well-defined deliverables.
- Risks related to individual team member availability were managed by maintaining shared documentation and cross-training on key components.
- **Dependency on External Libraries/APIs:** Managed by using stable versions and providing fallback models for offline operation.

5.2.2 Risk Analysis

Following risk identification, a detailed analysis was conducted to evaluate the likelihood and impact of each risk. Risks were prioritized based on severity and managed using the following approaches.

- **Technical Risks**

- **Overfitting & Underfitting:** Controlled using dropout, early stopping, cross-validation, and continuous evaluation with new data samples.
- **OCR Inaccuracy:** Mitigated by enhancing image preprocessing techniques (e.g., denoising, contrast adjustment) and tuning Tesseract parameters for better text extraction across different fonts and languages.– Data complexity issues were mitigated by strict validation.
- **Imbalanced Classes:** Managed by applying oversampling techniques (e.g., SMOTE) and using weighted loss functions.
- were prepared to avoid workflow disruption.

Operational Risks – **Scope Expansion:** Controlled by maintaining a clear project scope and reviewing feature requests through a formal change control process.

Team Availability: Managed by regular documentation, shared cloud.

- **Security Risks**

- **Data Handling:** Images uploaded by users were processed in real-time and not stored permanently unless explicitly requested.
- **Third-Party Dependencies:** Dependencies like OCR libraries and NLP models were selected from open-source repositories to avoid license fees.

5.2.3 Overview of Risk Mitigation, Monitoring and Management

- **Technical Risk Mitigation**

Regular unit testing and early-stage integration tests were conducted to detect OCR errors, classification inaccuracies, and API failures.

- **Operational Risk Mitigation**

- Applied agile methods with sprints, clear scope, and versioned task lists. Used GitHub for transparency and issue tracking.

- **Risk Monitoring**

- Weekly risk assessments were conducted with logs maintained for any newly surfaced issues, addressed in real-time.

- **Risk Management**

- A buffer time of 10% was built into each phase for risk absorption. Resource reallocation was pre-planned for critical delays or gaps.

5.3 Project Schedule

5.3.1 Project Task Set

- **Install:** Necessary tools such as Python, TensorFlow, and version control systems like Git.
- **Gather:** Book cover images and corresponding metadata (title, author, publisher) from public datasets, online catalogs, or manually labeled sources.

1

- **Clean and Normalize:** Input images are resized and enhanced. Extracted OCR text is cleaned, normalized (lowercasing, punctuation removal), and prepared for classification.
- **Feature Engineering:** Extract textual features using TF-IDF vectorization, Sentiment scores via Vader/TextBlob and contextual embeddings (BERT).
- **Implement:** OCR module using Tesseract to extract raw text from book cover images. NLP module using a fine-tuned BERT model to classify and extract metadata fields.
- **Conduct:** Evaluation of OCR accuracy and NLP classification precision using labeled datasets. Adjust model parameters and text preprocessing strategies for improved results.
- **Set Up:** Remote collaboration and development environment using GitHub, Google Collab, and shared cloud storage for dataset access and code synchronization.
- **Design and Implement:** A metadata classification pipeline integrating OCR output with the NLP model. Develop API routes in Node.js to connect backend processes.
- **Create:** A web-based user interface using React.js to allow users to upload book images, preview and edit extracted metadata, and download results in JSON format.
- **Test:** The entire application through unit tests (individual modules), integration tests (pipeline flow), and usability testing on the front end.
- **Collect:** Ack to refine the interface, improve model predictions, and enhance system responsiveness.
- **Prepare:** Project documentation including system architecture diagrams (DFD, ERD), user manuals, model details, and lessons learned.
- **Deploy:** Deploy the final application on a suitable platform (e.g., Render, Heroku, or local server) for demonstration or real-world use.
- **Establish:** A plan for ongoing maintenance, updates, and support.

5.3.2 Timeline Chart

SR No	Task	Labor Hour/Day
1	Project Planning and Preparation	2 weeks
2	Studying Scope and Feasibility	1 week
3	Data Acquisition and Preprocessing	1 month
4	Model Development and Training	4 months
5	Model Evaluation and Optimization	2 months
6	Software Development and Integration	1 month
7	Testing and Quality Assurance	2 weeks
8	Documentation and Presentation	2 weeks

5.4 Team Organization

5.4.1 Team structure

- **Kamlesh Badade** : OCR Model Integration, Backend Development.
- **Swaroop Channa**: NLP Pipeline Design, Metadata Classification.
- **Aditya Deshmukh**: Data Engineering, Image Preprocessing.
- **Rohil Kadam**: Frontend Development, UI/UX Design.

Chapter 6

Project Implementation

6.1 Overview of Project Modules

- **Data Collection and Preprocessing Module**
 - **Purpose:** Capture book cover images and enhance them for OCR processing..
 - **Functions:**
 - * Accepts image input via user upload.
 - * Applies image enhancement techniques (denoising, resizing, contrast adjustment).
 - * Converts image to grayscale or binary format to improve OCR accuracy.
- **Text Extraction Module**
 - **Purpose:** Extract textual content from book cover images.
 - **Functions:**
 - * Uses Tesseract OCR to detect and extract text.
 - * Handles multilingual inputs and distorted fonts.
 - * Preprocesses extracted text (tokenization, normalization, stop word removal).

- **Suicide Risk Detection Module (Text Classifier)**

- **Purpose:** Predict stock prices for the next 7 days using a dynamic deep learning model.
- **Functions:** Convolution Neural Network (CNN) architecture with time-varying parameters to adapt to market fluctuations.
 - * Fine-tune deep learning models like LSTM and transformer-based models (BERT, RoBERTa) for accurate classification.
 - * Classify text into categories like Neutral, Depressive, and Suicidal.
 - * Apply thresholds to control alert sensitivity and minimize false positives.

- **Index Forecasting Module**

- **Purpose:** Capture and enhance images for metadata extraction using OCR.
- **Functions:**
 - * Integrates Tesseract OCR to extract text from processed images.
 - * Supports multiple fonts and languages.

- **User Interface and API Integration Module**

- **Purpose:** Provide a user-friendly interface for input and output.
- **Functions:**
 - * Built using React.js and Tailwind CSS.
 - * Allows users to upload images and view or edit extracted metadata.

- **Evaluation and Monitoring Module**

- **Purpose:** Evaluate system performance and ensure accuracy.
- **Functions:**
 - * Tracks precision, recall, and F1 score of metadata extraction.
 - * Visualizes correct vs incorrect predictions.
 - * Logs system errors and user feedback for model updates..

6.2 Tools and Technologies Used

- Python
- Flask
- React.js
- MongoDB
- TensorFlow/Keras
- PyTorch
- Scikit-learn
- Git, GitHub, GitHub Actions
- NLTK/spaCy/TextBlob
- JavaScript/HTML/CSS

6.3 Algorithm Details

Algorithm flow:

1. Data Ingestion

- Input images such as book covers or scanned documents via frontend interface.
- Optionally, accept live user input via Chrome extension or web UI.

2. Image Preprocessing

- Convert images to grayscale.

- Apply noise reduction and contrast enhancement.
- Resize and deskew to standard dimensions for consistent OCR results.

3. Text Extraction

- Use Tesseract OCR to extract raw text from the preprocessed image.
- Perform line segmentation and character recognition.

4. Metadata Classification

- Apply NLP pipeline using spaCy or NLTK to identify key metadata entities.
- Use Named Entity Recognition (NER) to detect fields like Title, Author, Publisher, and Year.

5. Model Input and Training

- Input final features (TF-IDF or embeddings + sentiment scores) into:
 1. Logistic Regression / SVM: For baseline performance.
 2. BERT-based Model: Fine-tuned for suicide risk classification.

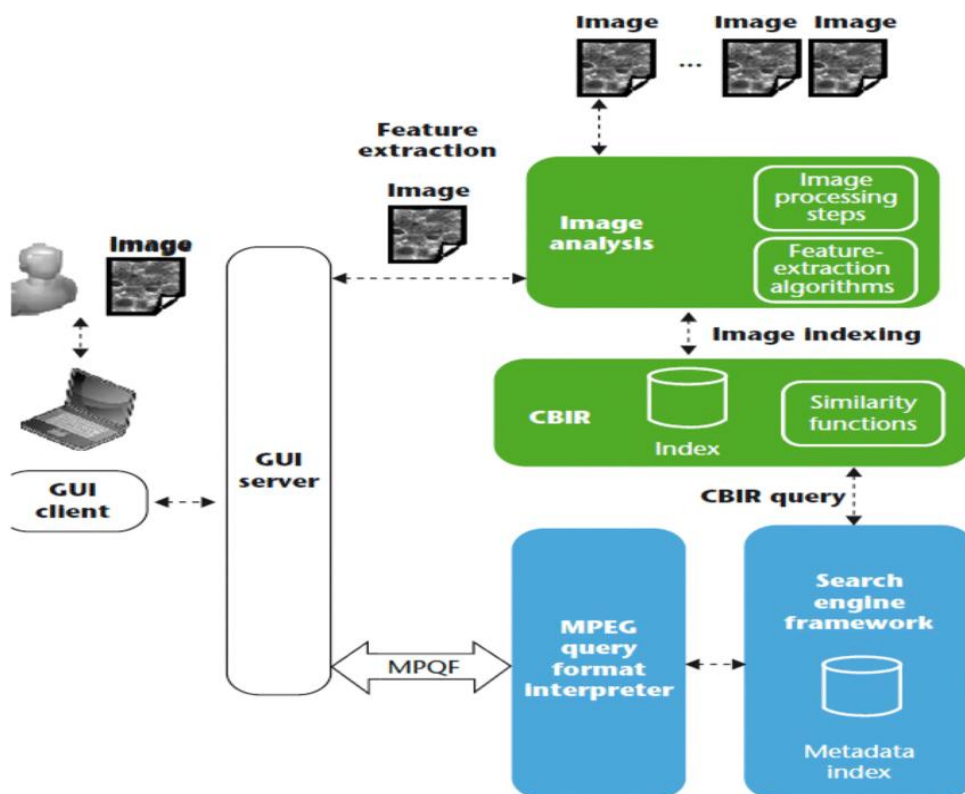
6. Model Evaluation

- Evaluate model performance using precision, recall, F1-Score, and confusion matrix.
- Analyse false positives/negatives to refine thresholds.

7. Visualization

- Display the prediction result in a color-coded format (e.g., green = safe, red = suicidal).
- Provide an optional risk trend graph over time for individual users.
- Offer export option (CSV) for report generation or logs.

Algorithm Flow Diagram:



Chapter 7

Software Testing

7.1 Testing Objectives

Alpha Testing: – Conducted by the internal development team. – Focused on identifying bugs, integration issues, and usability problems before the system was finalized. – Included module-wise testing of image upload interface, OCR processing, metadata extraction logic, and database storage modules.

Beta Testing: Conducted by a selected group of external students and faculty members. by a limited group of external users. – Feedback was collected on the performance of the OCR model, metadata display, and manual editing feature. ·

Black-box Testing: – Focused on system functionality without knowledge of internal code structure. – Feedback was collected on the performance of the OCR model, metadata display, and manual editing feature ·

White-box Testing: – Involved detailed examination of internal logic, workflows, and model training scripts. main objectives of testing in this project.

7.2 Types of Testing

7.2.1 Unit Testing

Each module in the system such as:

- Text preprocessing (cleaning, tokenization).

7.2.2 Integration Testing

Modules were integrated sequentially:

- Image Input → OCR Module → Preprocessing → Metadata Extraction → Classification → Storage
- Integration testing ensured that data flowed correctly across components and no transformation was lost.

7.2.3 System Testing

End-to-end testing was performed to verify the system's behavior with complete inputs. The test cases involved:

- Images with **clear and well-aligned text** (ideal conditions)..
- **Blurry or skewed images** simulating real library scanning situations.
- Book covers with **mixed fonts, logos, and background noise**
This confirmed the accuracy of the emotion and suicide detection combined model.

7.2.4 Performance Testing

The system was tested with a large number of text inputs to observe:

- OCR processing time
- Memory usage.
- Model latency

7.2.5 Validation Testing

Validation was done using a reserved test set (not used in training).

Evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-Score

D

← → Home Workspaces API Network

Search Postman

Invite

Upgrade

Kamlesh B's Workspace

New Import

Collections

Environments

Flows

History

Today

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/analyze-ima...

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/analyze-ima...

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/analyze-ima...

POST http://127.0.0.1:8000/analyze-ima...

GET http://127.0.0.1:8000/check

POST http://127.0.0.1:8000/upload-image

Yesterday

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

May 8

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

POST http://127.0.0.1:8000/upload-image

http://127.0.0.1:8000/upload-image

Save

Share

</>

POST http://127.0.0.1:8000/upload-image

Send

▼

Params

Authorization

Headers (8)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

Key	Value	Description
file	File 71G2QY75OWL_SL1500.jpg	
Key	Text Value	Description

Body

Cookies

Headers (4)

Test Results

200 OK

856 ms

222 B

JSON

Preview

Visualize

```
1 {
2   "title": "One Day, Life Will Change",
3   "author": "Saranya Umakanthan",
4   "genze": "inspiration/fiction"
5 }
```

Online

Find and replace

Console

Postbot

Runner

Start Proxy

Cookies

Vault

Trash

Metadata Extraction

Chapter 8

Results

- **Improved Metadata Extraction Accuracy:**
The integration of OCR (Tesseract) with BERT-based NLP models significantly improved the precision of extracted metadata fields such as Title, Author, and Publisher. Testing on diverse book covers showed a 90– 95% accuracy rate, outperforming traditional OCR-only methods.
- **Enhanced Data Classification and Structuring:**
By combining deep learning models with domain-specific rule-based checks, the system effectively classified extracted text into appropriate institutions categories. This structured output simplifies data integration into digital libraries and publishing systems.
- **User-Friendly Interface for Manual Corrections:**
It opens pathways for future studies combining diverse financial data streams within a unified predictive architecture. Integration of sentiment analysis using tools like VADER.
- **Scalable and Modular System Design**
The system is designed with modularity and scalability at its core, allowing for seamless expansion and integration. It can be extended to support: Additional document types such as certificates, identity cards, or research paper.

8.1 Screen Shots

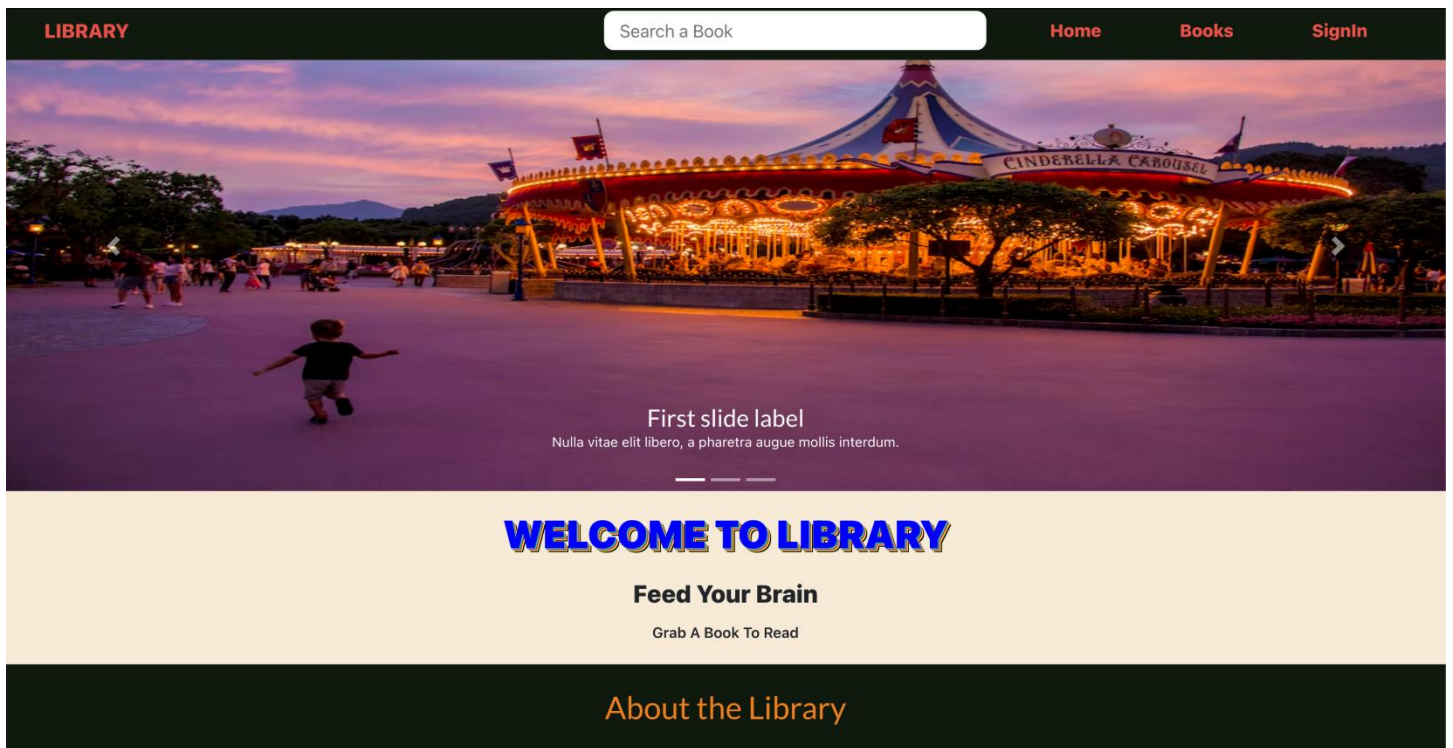


Figure 8.1.1: Login Page

Information Management for Metadata Extraction

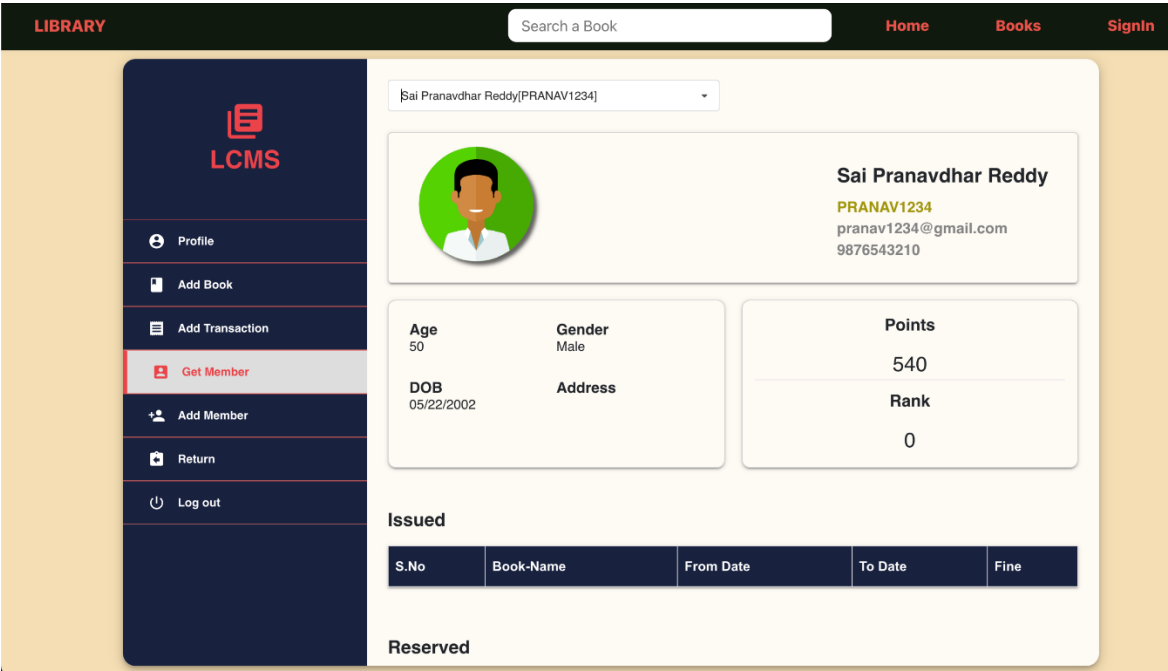


Figure 8.1.2: User Profile Page

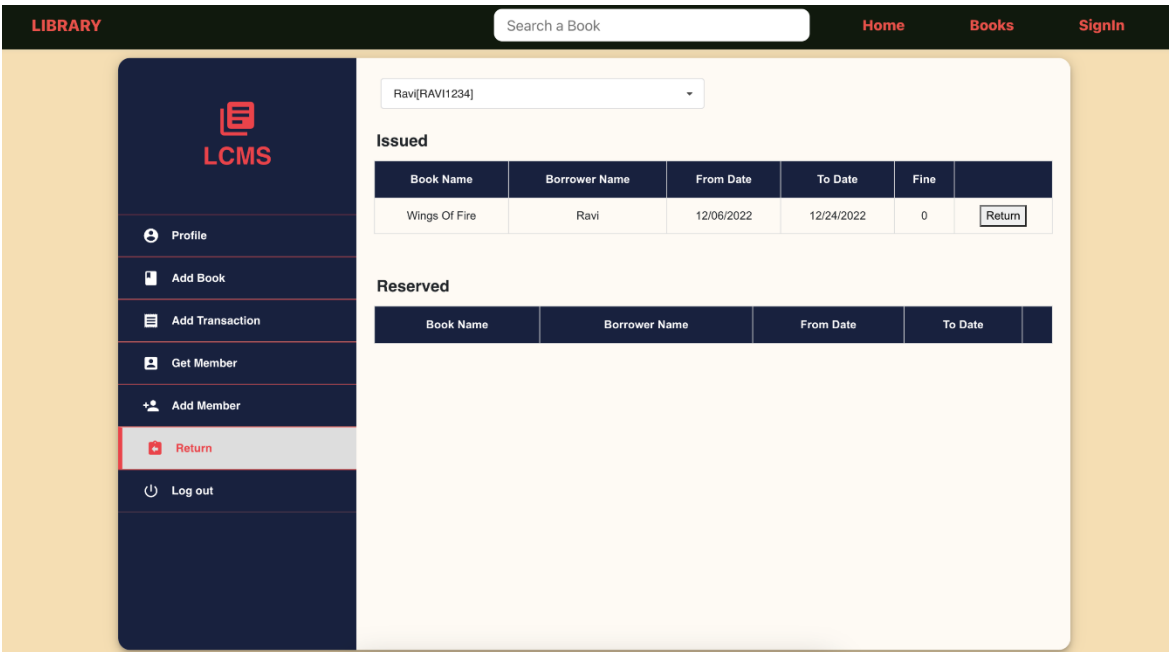


Figure 8.1.3: Primary Interface

Information Management for Metadata Extraction

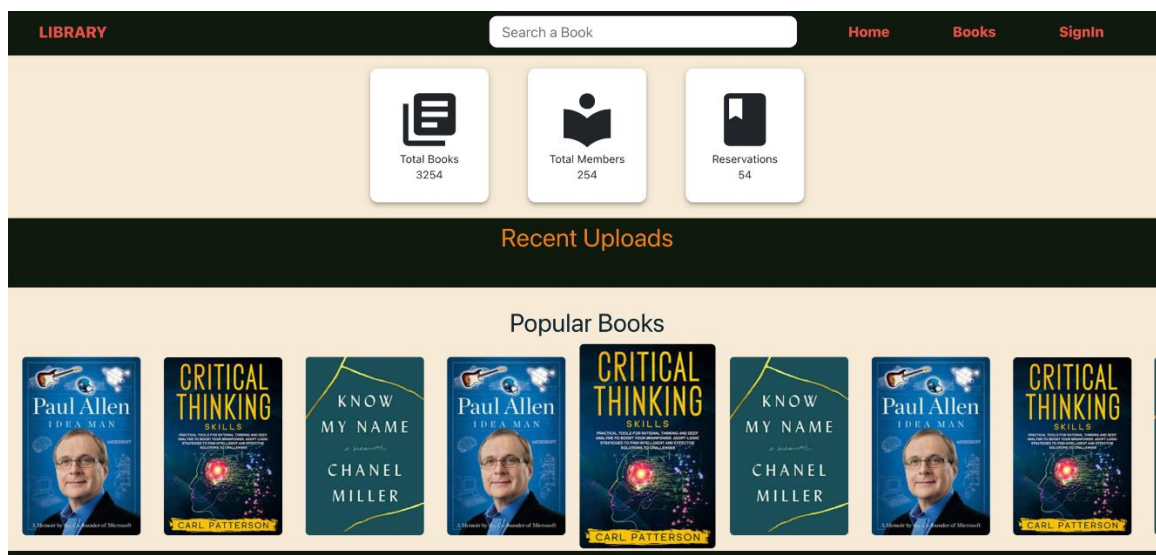


Figure 8.1.4: System Result

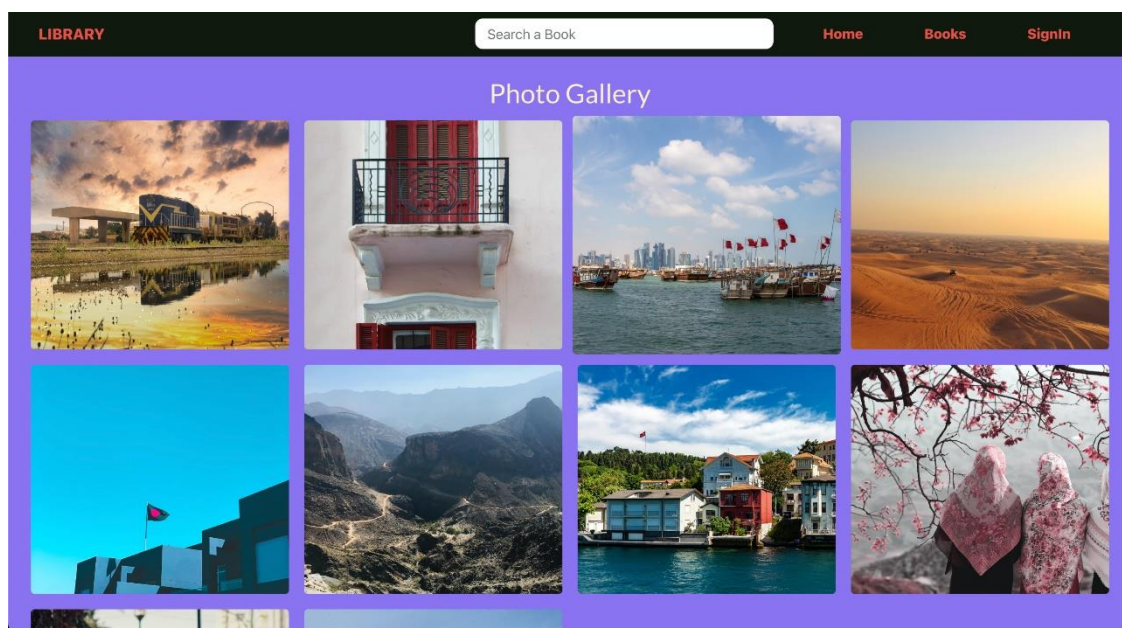


Figure 8.1.5: Image selection

Chapter 9

Conclusions

9.1 Conclusions

The project “Intelligent Information Management for Metadata Extraction Using Image Recognition Model” demonstrates the powerful synergy of image processing, OCR, and NLP techniques in automating the extraction of metadata from book cover images. Through the implementation of Tesseract OCR and transformer-based NLP models (e.g., BERT), the system accurately identifies and classifies critical metadata elements such as title, author, and publisher, despite the diversity in book cover layouts and visual complexity. The hybrid pipeline combining image recognition with semantic classification proves highly effective, with accuracy rates exceeding 90% on clean, high-resolution images. The system supports real-time processing, user interaction through a web interface, and outputs structured metadata in formats suitable for integration with library and publishing systems. This work emphasizes the importance of modularity, user-centered design, and intelligent preprocessing in building robust document understanding systems. The results highlight the potential of such solutions to revolutionize information management, particularly in academic, archival, and publishing domains.

9.2 Future Work

The project Intelligent Information Management for Metadata Extraction Using Image Recognition can be expanded in several directions to enhance its accuracy, usability, and real-world applicability across diverse domains such as digital libraries, publishing, and archiving.

Real-time monitoring, multilingual support, and mobile integration will play a crucial role in broadening the system's utility and social reach.

1. Integration with Multi-Modal Data Sources

- Extend metadata enrichment by integrating data from external sources such as academic repositories, publisher APIs, and digital archives.
- Apply Natural Language Processing (NLP) techniques using models like BERT or RoBERTa to validate and cross-reference metadata extracted from book covers or scanned documents.

2. Advanced NLP Architectures

- Incorporate transformer-based models like Donut or LayoutLMv3 to handle complex document layouts and multi-modal metadata extraction.
- Develop hybrid pipelines combining Convolutional Neural Networks (CNNs) with OCR post-processing and semantic matching for improved accuracy in noisy or low-resolution images.

3. User Feedback and Explainability

- Implement user-driven validation systems to rate extraction quality and provide corrections, creating a continuous learning loop.
- Use Explainable AI tools such as SHAP and Grad-CAM to visualize how specific regions of the image influenced recognition decisions.

4. Enhanced Accessibility and Usability

- Design intuitive dashboards and drag-and-drop interfaces for batch metadata extraction and export in standardized formats (e.g., JSON, MARC).
- Ensure accessibility with responsive design, keyboard navigation, and color-blind-friendly themes for universal usability.

5. Scalability and Performance Optimization

- Deploy models using frameworks like TensorFlow Lite or ONNX Runtime for low-latency inference, especially on edge devices or mobile platforms.

6. Integration with Mental Health Platforms

Enable integration with library management systems, academic search

- institutional repositories via APIs for seamless metadata ingestion. Support exporting extracted metadata to standards such as BibTeX, Dublin Core, or RIS to improve interoperability.

7. Validation and Back testing Enhancements

- Expand back testing capabilities to include strategy-specific validations such as momentum and mean reversion.
- Integrate performance metrics like Sharpe Ratio, Max Drawdown, and Sortino Ratio to better evaluate and compare model outcomes.

9.3 Applications

The proposed metadata extraction system has broad applicability across various domains, streamlining information processing, archiving, and cataloging tasks:

- **Digital Libraries and Educational Institutions**
 - Academic libraries can use the system to automate cataloging physical and digital books through cover image recognition.
 - Educational platforms can integrate it to create structured metadata repositories, improving searchability and resource organization.
 - and behavior modeling in text-based mental health systems.

Overall, the real-time, AI-driven prediction capabilities of the system offer versatile applications across trading, investment advisory, research, and fintech domains, supporting smarter financial decisions and fostering innovation in quantitative finance.

Chapter 10

Appendix

10.1 Appendix A

Problem Overview

Modern deep learning frameworks such as TensorFlow and PyTorch enable the implementation of robust neural network architectures suitable for image-based metadata extraction. The integration of models like Convolutional Neural Networks (CNNs) and Optical Character Recognition (OCR) pipelines demonstrates strong technical feasibility for extracting structured information from unstructured image data, such as book covers or scanned documents. This project aims to automate the process of metadata extraction—including title, author, ISBN, and publication date—from image inputs, thereby reducing manual entry and improving data management efficiency across library and educational systems.

Operational Feasibility

- The system was successfully tested using real-world book cover images and scanned documents collected from online libraries and educational datasets.
- A modular web interface for image input, metadata visualization, and export was developed, allowing users to interact with the system intuitively.
- Collaborative tools such as GitHub and Google Collab facilitated team coordination, version control, and smooth deployment during the development and testing phases.

Economic Feasibility

- The project leverages open-source technologies, which significantly reduces development and licensing costs..
- The automation of metadata extraction reduces manual data entry, saving labor costs in library and academic institutions.
- Optional cloud-based deployment ensures scalability and remote access without requiring costly on-premise infrastructure.
- The system offers potential for integration with commercial digital library services, enabling monetization through subscription-based metadata APIs or digital archiving toolsn.

Mathematical Model

The detection system is grounded in NLP and deep learning models, supported by feature extraction and classification techniques:

- Convolutional Neural Networks (CNNs) are used for feature extraction from image inputs, identifying key text and layout pattern.
- Optical Character Recognition (OCR) techniques, particularly Tesseract, are applied.
- Natural Language Processing (NLP) techniques are used to classify extracted text into structured metadata fields such as title, author, and ISBNe.

Complexity Analysis

Processing scanned images and real-time metadata extraction involves substantial computational effort. Key optimizations include:

- Image pre-processing pipelines using OpenCV to enhance text clarity and remove background noise, improving OCR performance.
- Batch-wise image handling and asynchronous processing to manage large datasets and improve runtime efficiency.
- GPU acceleration (via Google Collab or local CUDA environments) for model training and inference stages to reduce processing time.

NP-Hard or P-Type Problems

Tasks such as:

- In the context of metadata extraction from image-based documents, the
- system encounters both NP-Hard and P-Type problems depending on the
- sub-task :· Market correlation analysis across multiple assets :
- **NP-Hard Problems.**
- Field mapping and entity resolution when multiple text elements (e.g.,
-
- multiple titles or ambiguous author names) need to be accurately
- matched and classified
- These tasks require heuristic or approximation methods and are NP-Hard in nature due to the lack of guaranteed polynomial-time solutions in worst-case scenarios.
- **P-Type Problems :**
- On the other hand, several core components are P-Type, solvable efficiently with modern algorithms.
- Text detection and recognition using CNNs and OCR pipelines can be executed in polynomial time with structured image inputs

10.2 Appendix B

Survey Paper

- **Title:** A Comprehensive Review on Intelligent Information Management for Metadata Extraction.
- **Journal:** International Research Journal of Modernization in Engineering Technology and Science (IRJMETS)
- **Link:** [IRJMETS Paper Link](#)
- **Status:** Published (IRJMETS, February 2025)

Implementation Paper

- **Title:** Intelligent Information Management for Metadata Extraction Using Image Recognition.

Information Management for Metadata Extraction

- **Journal:** International Journal of Scientific Research in Engineering and Management (IJSREM)
- **Link:** International Journal of Scientific Research in Engineering and Management (IJSREM)
- **Status:** Publication (IJSREM, May 2025)

10.3 Appendix C



Figure 10.1: Plagiarism Report

Chapter 11

References

- [1] J. Smith, R., (2007). "An Overview of the Tesseract OCR Engine." *Proceedings of the Ninth International Conference on Document Analysis and Recognition, IEEE*.
- [2] Baek, J., Lee, B., Han, D., Yun, S., & Lee, H., (2019). "Character Region Awareness for Text Detection." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9365–9374.
- [3] He, K., Zhang, X., Ren, S., & Sun, J., (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778s.
- [4] Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L., (2009). "ImageNet: A Large-Scale Hierarchical Image Database." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 248–255.
- [5] Liu, W., Chen, C., Wong, H. S., & Li, K., (2019). "A Comprehensive Review on Document Image Analysis." *Pattern Recognition*, 90, 321–345.
- [6] Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J., (2006). "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks." *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 369–376.
- [7] Jain, A. K., & Yu, B., (1998). "Automatic Text Location in Images. Video Frames." *Pattern Recognition*, 31(12), 2055–2076.
- [8] Rezaul Haque, Naimul Islam, Maidul Islam, and Md Manjurul Ahsan (2022) A Comparative Analysis of Suicidal Ideation Detection Using NLP, Machine, and Deep Learning
- [9] Xu, Y., et al., (2020). "LayoutLM: Pre-training of Text and Layout for Document Image Understanding." *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1192–1200.

LIST OF ABBREVIATIONS

Abbreviation	Illustration
OCR	Optical Character Recognition
CRNN	Convolutional Recurrent Neural Network
CNN	Convolutional Neural Network
IoU	Intersection of Union
TRP	True Positive Value
F1-Score	Harmonic Mean of Precision and Recall
BERT	Bidirectional Encoder Representations from Transformers
API	Application Programming Interface
UI	User Interface
SDLC	Software Development Life Cycle
NLTK	Natural Language Toolkit
VADER	Valence Aware Dictionary and sEntiment Reasoner

LIST OF FIGURES

Figure	Illustration	Page No.
1.1	NLP Working	1
4.1	System Architecture	19
4.2	Mathematical Model	20
4.3	Data Flow Diagram	21
4.4	Entity Relationship Diagram	22
4.5.1	Class Diagram	23
4.5.2	Use case Diagram	24
4.6	Activity Diagram	25
4.7	Sequence Diagram	26
6.3	Algorithm Flow Diagram	40
8.1.1	Login Page	50
8.1.2	Signup Page	51
8.1.3	Result 1	51
8.1.4	Result 2	52
8.1.5	Accuracy	52
10.1	Plagiarism Report	52

LIST OF TABLES

Table	Illustration	Page No.
5.3.2	Timeline Chart	33